Building AI Systems

From Prompts to Intelligent Workflows — Day 4

Today we go beyond writing prompts. We'll explore how real-world AI systems are designed — combining language models, external data, memory, and tools into powerful, production-ready workflows.



DAY 4 AGENDA

What We'll Cover Today

01

Beyond Text: Multimodal AI

Images, audio, and video — not just words

02

Retrieval-Augmented Generation (RAG)

Giving AI access to fresh, real-world knowledge

03

Embeddings & Semantic Search

How machines understand meaning

04

From Prompts to Systems

Designing end-to-end AI pipelines

05

AI Orchestration Frameworks

LangChain, LangGraph, and MCP



4.1

Beyond Text: Multimodal AI

Modern AI isn't limited to reading and writing. **Multimodal AI** systems can understand and generate text, images, audio, and video — just like a human uses multiple senses to make sense of the world.

4.1 — MULTIMODAL AI

The Four Modes of Modern AI

Think of multimodal AI like a Swiss Army knife — each blade handles a different type of information. A user could speak a question, attach a photo, and get a written summary back — all in one interaction.



Text

Summarization, translation, Q&A, code generation. The foundation of most LLMs today.



Image

Generate images from descriptions (DALL·E, Midjourney) or understand images (GPT-4V, Gemini).



Audio

Speech-to-text (Whisper), text-to-speech, and real-time voice assistants like ChatGPT Voice.



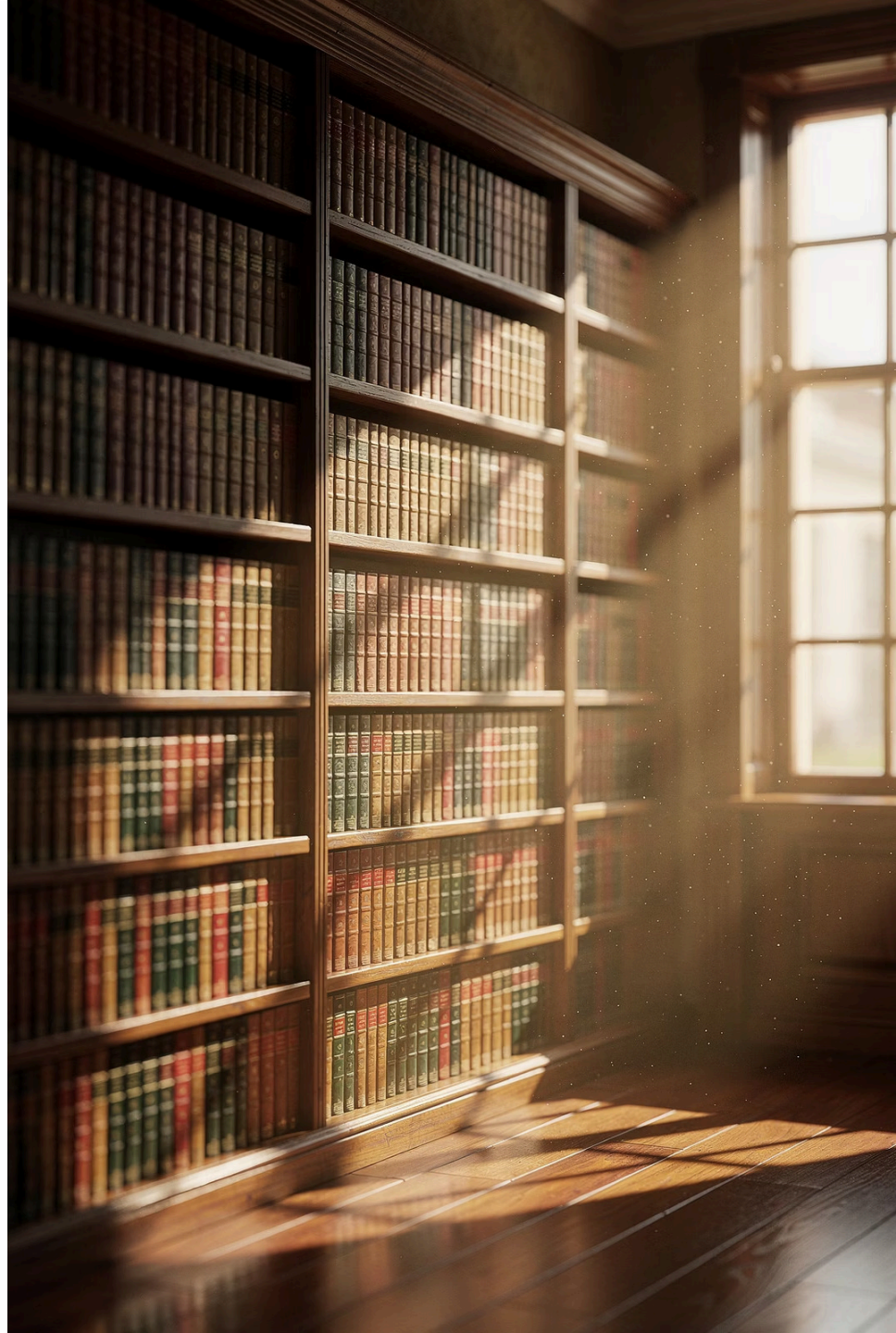
Video

Generate or analyze video content. Emerging models like Sora and Gemini 1.5 lead this frontier.

4.2

The Problem with LLMs: They're Frozen in Time

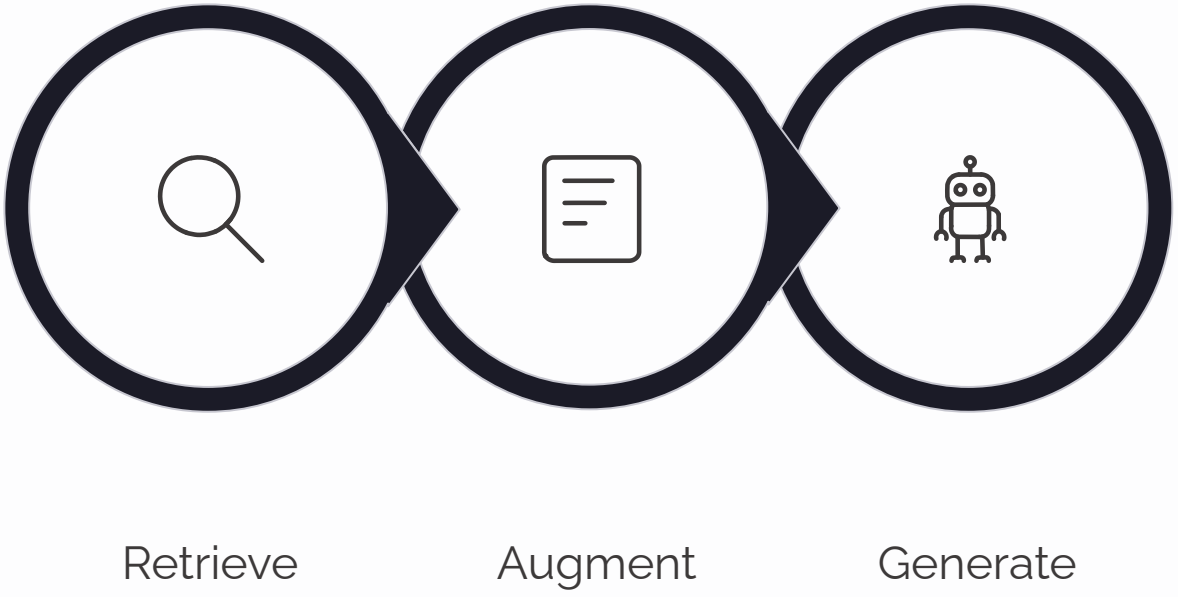
Imagine hiring a brilliant consultant who last read a newspaper in early 2024. They're smart — but they don't know about last month's product launch, your internal documentation, or yesterday's news. That's the core limitation of every LLM: a **knowledge cutoff**. This is where **Retrieval-Augmented Generation (RAG)** comes in.



4.2 — RAG

Retrieval-Augmented Generation (RAG)

RAG solves the knowledge problem by letting the AI **look things up before answering** — like a student who can reference notes during an open-book exam. Instead of relying only on what it memorized during training, the model retrieves relevant documents at query time, then uses that context to generate a grounded, accurate response.



This pattern is the backbone of AI assistants that reference company wikis, support docs, legal filings, or any dynamic knowledge base — keeping answers accurate and up-to-date without retraining the model.

4.3

Embeddings & Semantic Search

What Are Embeddings?

An embedding converts text into a list of numbers — a **vector** — that captures its meaning. Similar sentences end up with similar numbers, even if they use different words.

Example: "The car broke down" and "My vehicle stopped working" will have nearly identical vectors.

Why Does This Matter?

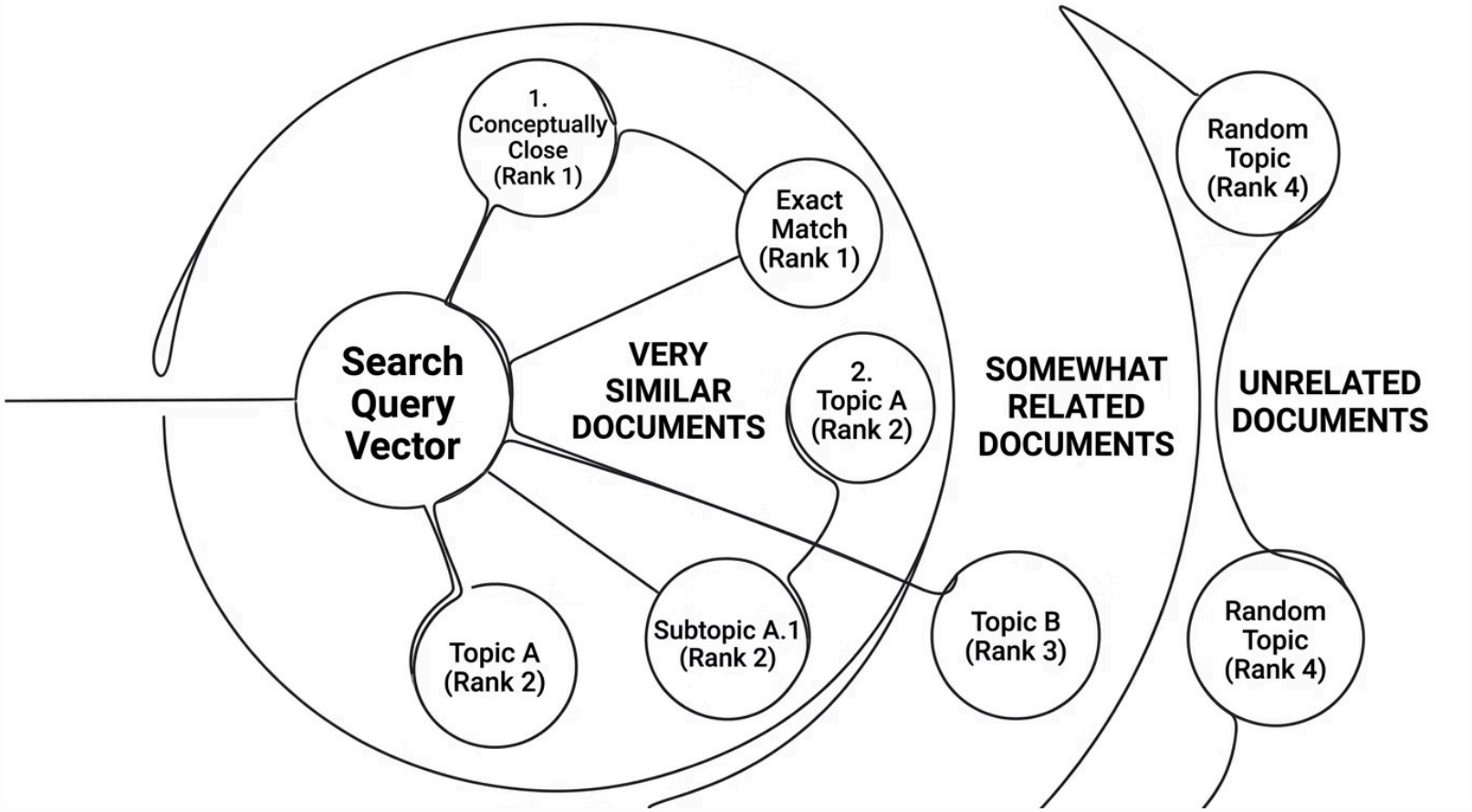
Traditional search looks for **exact keyword matches**. Semantic search uses embeddings to find results based on **meaning and intent** — just like how a great librarian understands what you're really looking for.

- Search "affordable housing" → also finds "low-cost apartments"
- Search "dog food" → also surfaces "pet nutrition" results
- Power RAG systems to retrieve the **most relevant** documents

4.3 — SEMANTIC SEARCH

Vector Search: Finding Meaning, Not Just Keywords

Think of a vector database like a vast map where every document occupies a location based on its meaning. When you search, you find the nearest neighbors on that map — the most semantically similar content, regardless of exact wording.



This is how RAG systems surface the **right chunks of information** from thousands of documents in milliseconds — making AI responses feel remarkably relevant and context-aware.



4.4

Why a Single Prompt Isn't Enough

A prompt is a question. A system is a **factory**. When you're building real AI products — a customer support bot, a coding assistant, an automated research tool — a single prompt can't handle authentication, memory, error recovery, tool use, and multi-step reasoning all at once. You need a **designed workflow**.

4.4 — AI SYSTEMS

The Four Core Components of an AI System

Every production AI system — no matter how complex — is built from the same four building blocks. Think of it like a restaurant: the chef (LLM) needs a menu (data), kitchen tools (tools), and the ability to remember regulars (memory).

🧠 LLMs

The reasoning engine. Takes inputs, understands context, and generates outputs. The "brain" of the system.

📂 Data Sources

Databases, documents, APIs, and knowledge bases the model can retrieve from. Keeps the AI informed and grounded.

🔧 Tools

External capabilities: web search, calculators, code interpreters, calendar APIs. Lets AI take real-world actions.

💾 Memory

Short-term (conversation context) and long-term (user preferences, history). Makes AI feel continuous and personalized.

4.4 — AI PIPELINES

Designing AI Workflows

Instead of one big prompt, you chain smaller, focused steps — each with a clear job. This is called an **AI pipeline**. It's the same principle behind assembly lines: reliable, testable, and easy to debug.

1

User Input

User asks a question or provides a task via chat, API, or UI

2

Retrieve

Fetch relevant documents or data from vector store or database

3

Reason

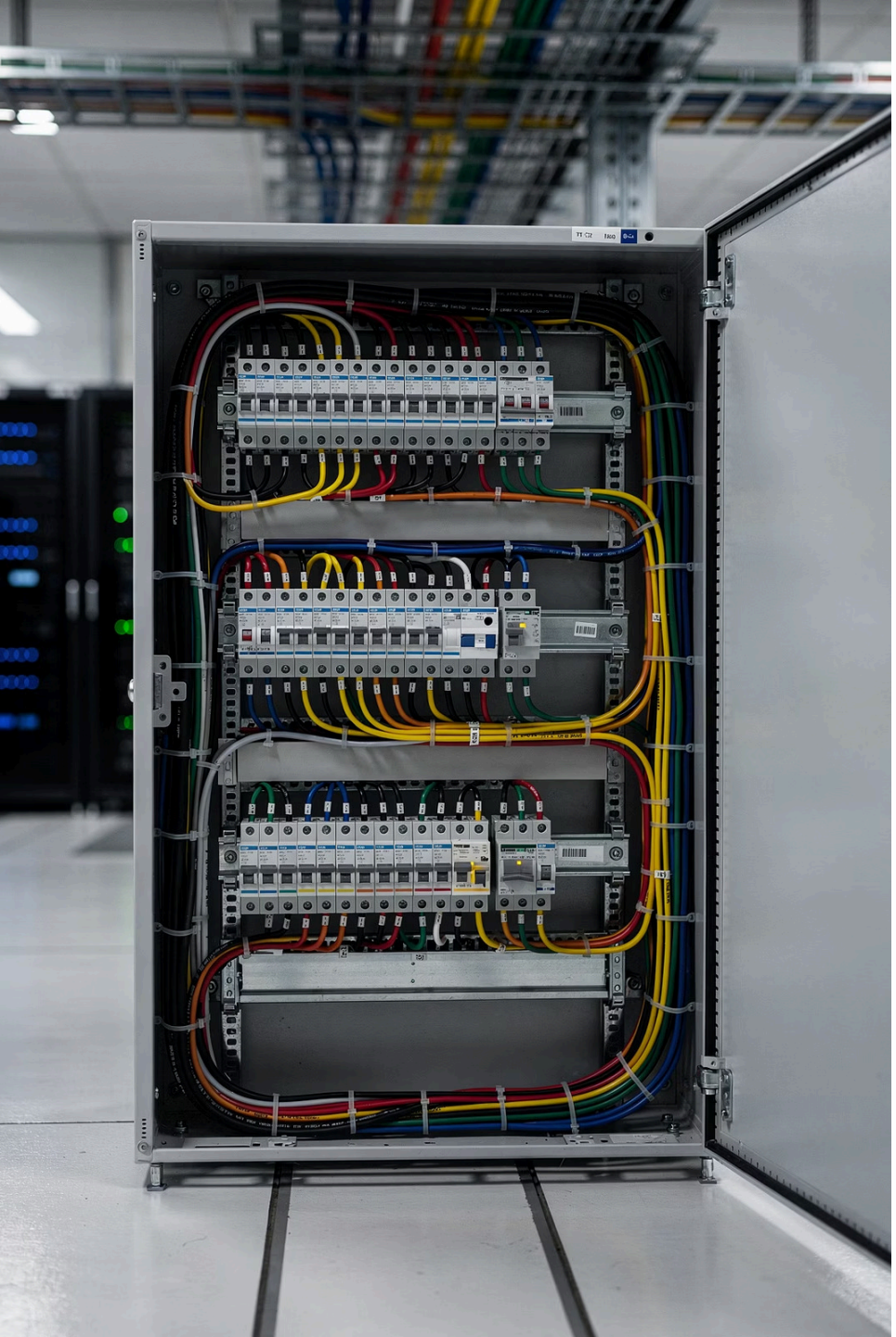
LLM processes context, calls tools if needed, and plans a response

4

Respond

Final answer is generated, stored in memory, and returned to the user

Real-world systems often loop back — the LLM may decide it needs more information, call another tool, and re-reason before giving a final answer. This is called an **agentic loop**.



4.5

AI Orchestration Frameworks

Building AI pipelines from scratch is like wiring a house without an electrician's toolkit. **Orchestration frameworks** give you the wiring diagrams, connectors, and safety switches so you can focus on what your system does — not how to glue it together.

4.5 — FRAMEWORKS

Three Frameworks You Should Know



LangChain — Rapid Prototyping

The most popular framework for connecting LLMs with tools, APIs, and data sources. Think of it as **LEGO for AI systems** — hundreds of pre-built integrations (databases, search engines, APIs) that snap together quickly. Best for: prototyping, building standard RAG pipelines, and exploring integrations fast.



LangGraph — Stateful Multi-Step Workflows

An extension of LangChain that models AI workflows as a **graph of nodes and edges**. Each node is a step; edges define what happens next. Perfect for agents that loop, branch, retry, or run parallel tasks. Best for: complex agents, multi-step decision trees, and workflows requiring state management.



Model Context Protocol (MCP) — The Universal Adapter

MCP is an emerging **open standard** (by Anthropic) that defines how AI models communicate with external tools and services. Like USB-C for AI — one protocol to connect any model to any tool. Best for: standardizing tool integrations across different AI models and environments.

KEY TAKEAWAYS

From Prompts to Intelligent Systems

Today's session showed how individual AI capabilities combine into full-stack, production-ready systems. Here's the mental model to carry forward:

Multimodal AI expands the input/output surface

AI can now see, hear, and speak — not just read and write. Design systems that leverage the right modality for the job.

RAG solves the knowledge freshness problem

Don't retrain — retrieve. Connect your LLM to live data sources using the Retrieve → Augment → Generate pattern.

Embeddings power meaning-aware search

Semantic search is the engine behind RAG. Store and query knowledge by meaning, not just keywords.

Systems = LLMs + Data + Tools + Memory

A single prompt is a question. A system is a designed workflow. Combine the four core components intentionally.

Use frameworks — don't reinvent the wheel

LangChain for speed, LangGraph for complexity, MCP for standardization. Pick the right tool for your scale.